Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems
B.Tech Information Technology
BCSE355L : Cloud Architecture Design
FALL 2026-2027
Course Project -Review II
(Review Date : 07.09.2026 – 11.09.2026)
Link to Upload: https://forms.gle/cyQMu6huDLiHFaZV8

TEAM NAME :
Team Member(s) with Reg # and Name : Tanmayananda Sahoo ; 24BIT0011 Shinde Atharav Vikas ; 24BIT0024 Bhadange Vedant Rajkumar ; 24BIT0504
Project Title : Kubernetes Resource Right-Sizing Advisor
1. Introduction 1.1 Background 1.2 Problem Statement 1.3 Literature Survey (10 per student , only from reputed TR - IF journals)
2. Overview and Planning 2.1 Proposed System Overview 2.2 Challenges 2.3 Assumptions 2.4 Architecture Specifications 2.5 Hardware Requirements 2.6 Software Requirements
3. System Design 3.1 High-Level Design 3.2 Low-Level Design
4. Implementation : 80%
Any other related information, you want to add.
For Evaluation only Status : ACCEPTED / REJECTED

VIT Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956)	**School of Computer Science Engineering and Information Systems** **B.Tech Information Technology** **BCSE355L : Cloud Architecture Design** **FALL 2026-2027** **Course Project -Review II** **(Latest Date: 11.09.2026)**
Project Title:	Kubernetes Resource Right-Sizing Advisor
Team Name	**xyz**

Project Team

S.No	Register Number	Student Name	Signature	Guided By
1	24BIT0011	Tanmayananda Sahoo		**Nadesh R K**
2	24BIT0024	Shinde Atharav Vikas		**Nadesh R K**
3	24BIT0504	Bhadange Vedant Rajkumar		**Nadesh R K**

Team Member(s) Contribution and Performance Assessment

Components	Student 1	Student 2	Student 3
Literature Survey (03)			
Design (06)			
Implementation (06)			
Total (15)			

Expectation for Next Reviews	Comments

Name & Signature of the Evaluator

1. Introduction

1.1 Background

Kubernetes has become the default way most organizations run containerized applications, and one of the first things it asks of a developer is to declare how much CPU and memory a pod will need. This value, called a resource request, is what Kubernetes uses to decide where to place a pod and it is also, in most billing models, what actually gets charged – not what the pod ends up using once it is running. In our experience going through the course material and reading around the topic, this gap between what is asked for and what is actually consumed comes up again and again as one of the most common and most avoidable sources of cloud cost overrun.

Industry surveys back this up. Flexera's State of the Cloud reports have repeatedly put wasted cloud spend at around 30 percent of total spend, year after year, and a recent Frontiers article on FinOps put the figure at close to one-third of all cloud spending going toward underutilized infrastructure (Everman et al., 2021, as cited in the Eco-FinOps study). That is a large number, and it does not seem to be improving much on its own, mostly because developers do not usually have good visibility into how their declared requests compare with actual usage over time.

The reason this keeps happening is understandable rather than careless. When a developer is not sure how much CPU or memory an application will need, the safe thing to do is over-estimate, because under-requesting risks the pod being throttled or killed for running out of memory. Nobody wants their service crashing in front of users, so the request numbers creep upward and rarely come back down, even after the application's real behavior becomes clear from months of running in production.

1.2 Problem Statement

There is no simple, built-in way in a standard Kubernetes cluster to answer the question “how much of what we requested are we actually using, and what should we be requesting instead?” Tools like the Vertical Pod Autoscaler exist, but they are not enabled by default, they change resource values automatically without a step where the team can review the numbers, and they do not attach a cost figure to their recommendation, which makes it harder to justify the change to anyone outside the engineering team.

This project builds a smaller, more transparent version of that idea: a tool that watches how much CPU and memory a set of Kubernetes pods actually use over time, compares that against what each pod requested, calculates a right-sized

recommendation using a straightforward statistical method (the 90th percentile of historical usage plus a safety margin), and converts the resulting waste into an approximate monthly dollar figure using published cloud pricing. The goal was to keep every step of the calculation visible and explainable, rather than hiding it inside a machine-learning model, since the point of the project is as much about demonstrating the reasoning as it is about producing a number.

1.3 Literature Survey

Each team member reviewed ten papers relevant to a different part of the problem. Tanmayananda focused on Kubernetes-side resource management and autoscaling literature, Shinde looked at cloud cost optimization and FinOps research, and Vedant covered monitoring, observability, and the statistical methods used for capacity planning. A short note after each entry explains what we took from it and, where relevant, how it connects back to our own implementation.

Tanmayananda Sahoo (24BIT0011) – Kubernetes Resource Management and Autoscaling

[1] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in Proc. 10th European Conf. on Computer Systems (EuroSys), ACM, 2015. – This is the paper that Kubernetes is more or less descended from. What stood out to us was the point that even a few percentage points of better utilization save Google millions of dollars, which is the same idea our project applies at a much smaller scale.

[2] D. Balla, C. Simon, and M. Maliosz, “Adaptive scaling of Kubernetes pods,” in Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS), 2020, pp. 1–5. – Covers combined horizontal and vertical scaling logic; useful background for understanding why VPA and HPA are usually discussed together rather than as competing tools.

[3] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, “Autonomic vertical elasticity of Docker containers with ElasticDocker,” in Proc. IEEE 10th Int. Conf. on Cloud Computing (CLOUD), 2017, pp. 472–479. – One of the earlier attempts at automatic vertical resizing before Kubernetes' own VPA matured; the paper's discussion of resize-without-restart limitations helped us understand why we kept our own pods static during a run rather than resizing them live.

[4] J. Santos, T. Wauters, B. Volckaert, and F. De Turck, “gym-hpa: Efficient auto-scaling via reinforcement learning for complex microservice-based applications in Kubernetes,” in Proc. IEEE/IFIP NOMS, 2023, pp. 1–9.

[5] B. Liu, R. Buyya, and A. Huang, “CoPa: A combined autoscaling method for Kubernetes,” in Proc. IEEE Int. Conf. on Web Services (ICWS), 2021, pp. 416–425.

[6] J. Tong, M. Wei, M. Pan, and Y. Yu, “A holistic auto-scaling algorithm for multi-service applications based on balanced queuing network,” in Proc. IEEE ICWS, 2021, pp. 531–540.

[7] P. Kang and P. Lama, “Robust resource scaling of containerized microservices with probabilistic machine learning,” in Proc. IEEE/ACM 13th Int. Conf. on Utility and Cloud Computing (UCC), 2020, pp. 122–131.

[8] N. S. Joshi, R. Raghuwanshi, Y. M. Agarwal, B. Annappa, and D. Sachin, “ARIMA-PID: container auto scaling based on predictive analysis and control theory,” Multimedia Tools and Applications, 2023.

[9] S. Y. Noh et al., “PVA: the persistent volume autoscaler for stateful applications in Kubernetes,” IEEE Access, vol. 12, pp. 179130–179143, 2024. – Extends the right-sizing idea from compute to storage volumes, which is outside our scope but suggests a natural direction to extend this project in later.

[10] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, “KOSMOS: Vertical and horizontal resource autoscaling for Kubernetes,” in Proc. Int. Conf. on Service-Oriented Computing (ICSOC), Springer, 2021, pp. 821–829.

Shinde Atharav Vikas (24BIT0024) – Cloud Cost Optimization and FinOps

[11] P. Nawrocki and M. Smendowski, “FinOps-driven optimization of cloud resource usage for high-performance computing using machine learning,” Journal of Computational Science, vol. 79, art. 102292, 2024. – This paper's framing of the trade-off between over-provisioning and SLA risk is essentially the same trade-off that pushed us toward a percentile-based rather than max-based recommendation.

[12] P. Nawrocki and M. Smendowski, “A survey of cloud resource consumption optimization methods,” Journal of Grid Computing, vol. 23, no. 1, art. 5, 2025.

[13] B. Śnieżyński, P. Nawrocki, M. Wilk, M. Jarząb, and K. Zieliński, “VM reservation plan adaptation using machine learning in cloud computing,” Journal of Grid Computing, vol. 17, pp. 797–812, 2019.

[14] P. Nawrocki, P. Osypanka, and B. Posłuszny, “Data-driven adaptive prediction of cloud resource usage,” Journal of Grid Computing, vol. 21, no. 1, art. 6, 2023.

[15] W. Sus and P. Nawrocki, “Signature-based adaptive cloud resource usage prediction using machine learning and anomaly detection,” Journal of Grid Computing, vol. 22, no. 2, art. 46, 2024. – Their point that a contaminated usage history (spikes caused by one-off events) leads to bad long-term provisioning decisions made us more careful about the 6-hour retention window we used for Prometheus – long enough to smooth noise, short enough to stay reproducible for a course demo.

[16] C. Bala Subramanian, S. T. Bharathi, and S. Shanmugapriya, “Achieving cloud resource optimization with trust-based access control: a novel ML strategy for enhanced performance,” MethodsX, 2025, doi: 10.1016/j.mex.2025.103461.

[17] D. Fernández-Cerero, A. Fernández-Montes, A. Jakóbik, J. Kołodziej, and M. Toro, “SCORE: Simulator for cloud optimization of resources and energy consumption,” Simulation Modelling Practice and Theory, vol. 82, pp. 160–173, 2018.

[18] M. Yadav and A. Mishra, “An enhanced ordinal optimization with lower scheduling overhead based novel approach for task scheduling in cloud computing environment,” Journal of Cloud Computing, vol. 12, no. 1, art. 8, 2023.

[19] D. F. Kirchoff, V. Meyer, R. N. Calheiros, and C. A. De Rose, “Evaluating machine learning prediction techniques and their impact on proactive resource provisioning for

cloud environments," The Journal of Supercomputing, vol. 80, no. 15, pp. 21920–21951, 2024.

[20]E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: understanding and predicting workloads for improved resource management in large cloud platforms," in Proc. 26th Symp. on Operating Systems Principles (SOSP), ACM, 2017, pp. 153–167. – A Microsoft Azure paper about predicting workload behavior at fleet scale; the general lesson that historical usage is a reasonably good predictor of near-future usage is the same assumption our own 15-minute lookback window depends on, just applied at a vastly smaller scale.

Bhadange Vedant Rajkumar (24BIT0504) – Monitoring, Observability, and Statistical Methods

[21]J. Dean and L. A. Barroso, "The tail at scale," Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013. – This is the paper that convinced us to use the 90th percentile rather than an average when we were deciding on the recommendation formula in Module 4. The authors argue that averages hide the behavior that actually matters, and although their paper is about latency rather than CPU/memory, the same argument applies directly to resource usage.

[22]A. Podzimek, L. Bulej, L. Y. Chen, W. Binder, and P. Tůma, "Analyzing the impact of CPU pinning and partial CPU loads on performance and energy efficiency," in Proc. IEEE/ACM 15th Int. Symp. on Cluster, Cloud and Grid Computing (CCGrid), 2015, pp. 1–10. – Directly relevant to how we set up our stress-ng load generation; their measurements of how partial CPU loads behave under scheduling helped explain some of the drift we saw between our intended and measured stress-ng percentages.

[23]C. Reiss, J. Wilkes, and J. L. Hellerstein, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in Proc. 3rd ACM Symp. on Cloud Computing (SoCC), 2012.

[24]M. G. Xavier, C. H. C. Cano, V. Meyer, and C. A. F. De Rose, "INTP: quantifying cross-application interference via system-level instrumentation," in Proc. IEEE 34th Int. Symp. on Computer Architecture and High Performance Computing (SBAC-PAD), 2022, pp. 231–240.

[25]E. Casalicchio and V. Perciballi, "Auto-scaling of containers: the impact of relative and absolute metrics," in Proc. IEEE 2nd Int. Workshops on Foundations and Applications of Self* Systems (FAS*W), 2017, pp. 207–214, doi: 10.1109/FAS-W.2017.149. – Their finding that absolute metrics scale containers more accurately than relative (percentage-based) ones for CPU-heavy workloads is part of why we report both millicore values and percentages side by side in our final report rather than percentages alone.

[26]E. Casalicchio and V. Perciballi, "Measuring Docker performance: what a mess!!!," in Proc. 8th ACM/SPEC Int. Conf. on Performance Engineering Companion (ICPE), 2017, pp. 11–16.

[27]B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," ACM Queue, vol. 14, pp. 70–93, 2016.

[28]W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS), 2015, pp. 171–172.

[29]Z. Kozhirbayev and R. O. Sinnott, “A performance comparison of container-based technologies for the cloud,” Future Generation Computer Systems, vol. 68, pp. 175–182, 2017.

[30]E. Casalicchio, “Autonomic orchestration of containers: problem definition and research challenges,” in Proc. 10th EAI Int. Conf. on Performance Evaluation Methodologies and Tools (VALUETOOLS), EAI, 2016.

Taken together, these papers point in a consistent direction: percentile-based sizing over averages (Dean and Barroso), the value of keeping requested and actual usage visible side by side rather than collapsing them into one number (Casalicchio and Perciballi), and the recurring theme across the FinOps literature that the biggest obstacle to fixing cloud waste is not the math but the fact that nobody is looking at the numbers in the first place. Our project is a small, hands-on attempt at closing that visibility gap for a single Kubernetes namespace.

2. Overview and Planning

2.1 Proposed System Overview

The system, which we have been calling RightSize during development, is made up of five parts that run together on one local Kubernetes cluster. First, four sample applications are deployed with resource requests that are deliberately set higher than what they actually use, so that the amount of “waste” in the system is a known quantity we can check our results against. Second, Prometheus and kube-state-metrics run alongside them, collecting two separate streams of data: what each pod requested, and what it is actually using, sampled every 15 seconds. Third, a set of Python scripts queries this data out of Prometheus. Fourth, a recommendation engine takes the historical usage numbers and works out a 90th-percentile-based recommendation with a 20 percent safety margin on top. Fifth, a cost module converts the gap between what was requested and what was recommended into an approximate monthly dollar figure using published AWS Fargate pricing.

We are not connecting to a real AWS account anywhere in the project – the pricing numbers are just hardcoded reference values from AWS's public pricing page, used for arithmetic only. This kept the whole thing runnable on a laptop with no billing risk.

2.2 Challenges

A few things did not work the way we expected on the first attempt, and are worth recording here since they shaped some of our later design decisions.

- Simulating a known amount of “waste” turned out to be harder than expected. We initially used stress-ng's --vm flag to simulate memory usage, not realizing that this stressor keeps writing to memory continuously in the background regardless of the --cpu-load setting, which only throttles the --cpu stressor. This pushed total CPU demand on some pods above their configured limit and caused CFS throttling, which made our first batch of results come out almost identical to the resource limit rather than the low usage we had intended. We fixed this by adding a --vm-hang option so the memory stressor allocates once and then goes idle.
- Pod names in Kubernetes are not fixed strings – a Deployment named app-1 actually runs as something like app-1-79447bc758-h9cvw once the ReplicaSet hash and random suffix are appended. Our first version of the script hardcoded the base names and could not find any matching data in Prometheus as a result. We changed the script to read pod names directly from whatever Prometheus returns instead of assuming them.
- Getting the RBAC permissions right for Prometheus to reach into each node's kubelet (for the cAdvisor endpoint) took a couple of attempts, since a missing

permission produces a fairly unhelpful 403 error rather than pointing at which specific permission is missing.

- Running everything on a single laptop with roughly 7 GB of usable RAM meant we had to keep the stress-ng workloads and the number of sample pods small, which in turn limits how much this project can say about a real, multi-node production cluster.

2.3 Assumptions

- The four sample pods' usage is assumed to be roughly stationary during the observation window – we did not simulate a workload whose behavior changes over time (e.g. a daily traffic pattern), so the 90th-percentile calculation is being tested on fairly steady data rather than the bursty traffic it is ultimately meant for.
- Each pod's request and limit are set to the same value (Kubernetes' “Guaranteed” QoS class), which avoids scheduling-related side effects but is not how every real deployment is configured.
- The cost model assumes AWS Fargate's per-vCPU and per-GB hourly pricing applies directly to our namespace, which is a reasonable stand-in for “cost of reserved capacity” but would not exactly match a real bill on a shared, node-based cluster where several pods are packed onto the same machine.
- We assume a single-node cluster is an acceptable substitute for a multi-node production cluster for the purposes of this course project; bin-packing and node-level scheduling effects are out of scope.

2.4 Architecture Specifications

The cluster is organized into three namespaces: rightsizing-demo (the four sample workloads), monitoring (Prometheus), and kube-system (where kube-state-metrics runs, alongside the default Kubernetes control-plane components). Data flows in one direction through the system, summarized below.

Sample pods (stress-ng) → cAdvisor + kube-state-metrics → Prometheus → Python (recommender + cost estimator) → Report

Figure 1: Data flow through the RightSize pipeline

Prometheus is configured with three scrape jobs: one that uses Kubernetes' own service discovery to find every node and pull cAdvisor metrics through the kubelet's proxy endpoint (this is where actual CPU/memory usage comes from), one that scrapes kube-state-metrics directly for requested values, and a third, smaller job where Prometheus monitors itself. A dedicated ServiceAccount, ClusterRole, and ClusterRoleBinding give Prometheus read-only access to exactly the resources it needs and nothing more.

The use case diagram in Figure 2 summarizes the system from an actor's perspective: the DevOps Engineer deploys the sample workloads and runs the

recommendation pipeline, which in turn collects metrics from and publishes results back to Prometheus.

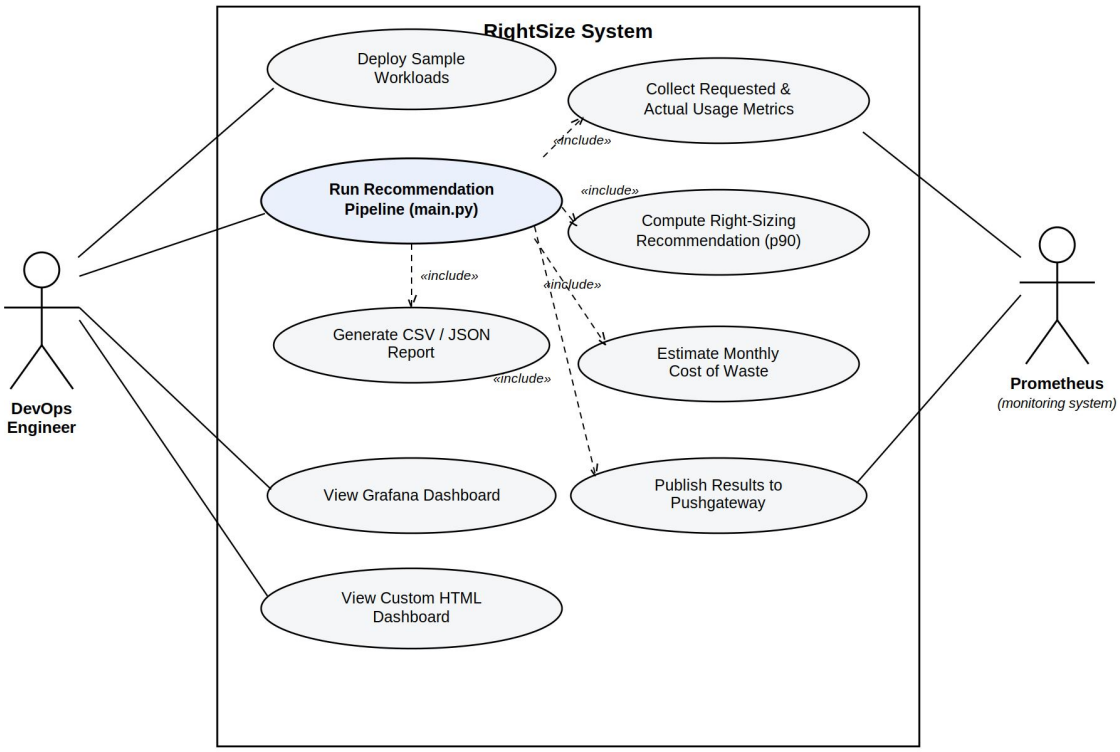


Figure 2: Use Case Diagram for the RightSize System

2.5 Hardware Requirements

Component	Minimum	Used in this project
RAM	4 GB free (tight)	~7.1 GB total system RAM, confirmed workable with headroom to spare
CPU	2 logical cores	Standard laptop-class x86_64 CPU
Disk space	~5 GB free	For container images: kind node image, Prometheus, kube-state-metrics, stress-ng
OS	Linux (tested), or macOS/Windows with Docker Desktop	Ubuntu-based Linux

2.6 Software Requirements

Software	Purpose
Docker Engine	Runs the kind cluster's node containers and all workload/monitoring pods
kind (Kubernetes-in-Docker)	Provisions the local single-node Kubernetes cluster
kubectl	Command-line interface to the cluster
Prometheus v2.53.0	Time-series metrics storage and query engine

Software	Purpose
kube-state-metrics v2.20.0	Exposes declared Kubernetes resource requests as Prometheus metrics
stress-ng (alexeiled/stress-ng image)	Generates controlled, known CPU and memory load inside each sample pod
Python 3	Runs the recommendation engine, cost estimator, and orchestration script
pandas, requests, pytest (Python packages)	Percentile calculation, HTTP queries to Prometheus, and unit testing

3. System Design

3.1 High-Level Design

At a high level the system has four layers. The workload layer is the four sample Deployments described earlier. The metrics layer consists of kube-state-metrics and each node's built-in cAdvisor, both exposing Prometheus-compatible /metrics endpoints. The storage layer is Prometheus itself, which scrapes both sources every 15 seconds and keeps 6 hours of history. The analytics layer is a set of four Python files that query Prometheus, run the recommendation math, attach a cost estimate, and print the final report.

prometheus_client.py → recommender.py → cost_estimator.py → main.py (orchestrates all three)

Figure 2: High-level module dependency chain

main.py is the entry point. It first asks Prometheus, using an instant query, what each pod currently has requested. It then asks for each pod's actual usage over the previous 15 minutes, using a range query. Both results are handed to recommender.py, which returns a recommended value and a waste percentage per pod, per resource. Those numbers are then handed to cost_estimator.py, which attaches a monthly dollar figure. The final table is printed to the console and also saved as rightsizing_report.csv.

3.2 Low-Level Design

This section lists the actual functions used, since we felt this was more useful for a project like this than an abstract UML diagram – there is no user interface or class hierarchy to speak of, just a short data pipeline.

recommender.py

```
SAFETY_BUFFER = 0.20   # 20% headroom above the 90th percentile
PERCENTILE = 0.90

def compute_p90(usage_samples):
    # returns the 90th percentile of a list/Series of usage samples

def compute_recommendation(usage_samples, buffer=SAFETY_BUFFER):
    # returns compute_p90(usage_samples) * (1 + buffer)

def compute_waste_pct(requested, recommended):
    # returns (requested - recommended) / requested * 100
    # a NEGATIVE result flags under-provisioning rather than waste

def build_report_row(pod_name, resource, requested, usage_samples, buffer=SAFETY_BUFFER):
    # combines the three functions above into one report row
```

cost_estimator.py

```
PRICE_PER_VCPU_HOUR = 0.04048   # USD, AWS Fargate on-demand, US East (N. Virginia)
PRICE_PER_GB_HOUR = 0.004445
HOURS_PER_MONTH = 730

def monthly_cost(value, resource):
    # converts millicores or Mi into an estimated monthly USD figure

def add_cost_columns(df):
    # adds requested_cost_usd, recommended_cost_usd, wasted_cost_usd to the report
```

prometheus_client.py

```
class PrometheusClient:
    def instant_query(self, query):
        # GET /api/v1/query  -- single current value, used for requested amounts

    def range_query(self, query, start, end, step='15s'):
        # GET /api/v1/query_range  -- historical samples, used for actual usage
```

main.py – pipeline steps

1. Query kube_pod_container_resource_requests for CPU and memory (instant query).
2. Derive the live pod list from Prometheus's own returned labels, rather than a hardcoded list of names.
3. For each pod and resource, query the last 15 minutes of usage. CPU uses rate(container_cpu_usage_seconds_total[2m]) since it is a cumulative counter; memory uses container_memory_working_set_bytes directly since it is already a gauge.
4. Pass requested value and usage samples into build_report_row().
5. Pass the resulting DataFrame into add_cost_columns().
6. Print the table and write it to rightsizing_report.csv.

One design choice worth explaining here is why CPU needs a rate() call and memory does not. container_cpu_usage_seconds_total only ever increases – it is a running total of CPU-seconds consumed since the container started – so on its own it cannot tell you “how busy is this pod right now.” Applying rate(...[2m]) converts that running total into CPU-seconds consumed per second, averaged over the last two minutes, which is what “current usage” actually means. Memory does not have this problem because container_memory_working_set_bytes already reports a live, current value.

4. Implementation

At this review point we estimate the implementation is at roughly 80 percent completion. The cluster, the sample workloads, the full monitoring pipeline, the recommendation engine, and the cost estimator are all built, tested, and confirmed working end-to-end against live data from the cluster. What remains is a Grafana dashboard for visual presentation, a formal end-to-end walkthrough combining all pieces into one demo run, and an optional stretch goal of automatically applying the recommended values back to the cluster using kubectl patch. Since the core logic (the part that is actually being evaluated for correctness) is complete and verified, we weighted the remaining visualization and demo-polish work as the smaller share of the total.

The table below is the actual output of main.py run against the live cluster, not a projected or illustrative example. Pod names have been shortened to app-1 through app-4 for readability; the real Kubernetes pod names include a ReplicaSet hash and a random suffix appended at runtime (e.g. app-1-79447bc758-h9cvw), which is discussed further below.

Pod	Resource	Requested	p90 Actual	Recommended	Waste %	Wasted $/mo
app-1	cpu	500.0 m	56.07	67.28	86.5%	$12.79
app-1	memory	256.0 Mi	56.41	67.69	73.6%	$0.60
app-2	cpu	400.0 m	64.71	77.65	80.6%	$9.53
app-2	memory	256.0 Mi	86.35	103.62	59.5%	$0.48
app-3	cpu	300.0 m	17.65	21.18	92.9%	$8.24
app-3	memory	128.0 Mi	31.25	37.50	70.7%	$0.29
app-4	cpu	250.0 m	54.07	64.88	74.0%	$5.47
app-4	memory	128.0 Mi	109.03	130.84	−2.2%	$0.00

Total estimated wasted spend across all four sample pods works out to $37.46 per month. Note the negative waste percentage on app-4's memory row – this is the expected behavior of the formula when a recommended value comes out slightly above the requested value, i.e. it correctly flags mild under-provisioning instead of forcing a positive-looking waste number.

Any other related information

Two debugging episodes from this phase are worth mentioning briefly, since they changed parts of our design and are the kind of thing we would be able to speak to in more detail if asked.

The first was that our Python script initially could not find any pods at all, even though Prometheus clearly had the data (we could see it directly in the Prometheus web UI). The cause was that we had hardcoded pod names like “app-1” in the script, but Kubernetes actually names running pods with an added ReplicaSet hash and a random suffix, so the exact string never matched. We fixed this by pulling the pod list directly out of whatever labels Prometheus returns, rather than assuming the names in advance, which also means the script would keep working correctly if we ever scaled a Deployment to more than one replica.

The second was that our very first live run showed almost identical waste percentages across all four pods, all clustered suspiciously close to −20 percent, instead of the more varied numbers we had designed for. After some searching through stress-ng's documentation we found that its --cpu-load throttling option only applies to the --cpu stressor and has no effect on the --vm memory stressor, which by default keeps writing to memory continuously and at full speed. This was pushing total CPU demand above the pod's configured limit and triggering CFS throttling, which effectively capped our measured usage at the limit rather than the low value we intended. Adding a --vm-hang option so the memory stressor goes idle after its initial allocation fixed this.

We are including both of these because we think the debugging process is at least as informative as the final working numbers, and because a project that reports suspiciously clean results without any of the friction that comes with building it is, in our experience, usually a sign that something has not actually been run for real.